

Auditoría de Diseño de Software

Sistema EIEInfo

Entrega 1: Levantamiento del Sistema y Diagnóstico Inicial

Mariana Bernal Hernández
C31121

IE0417 Diseño de Software para Ingeniería
Escuela de Ingeniería Eléctrica, UCR

I Ciclo 2026

Responder una pregunta central:

Pregunta guía

¿Qué sistema tenemos enfrente y cuáles son sus principales rasgos de diseño?

Para esto se revisó la arquitectura, módulos, dependencias, configuración, funcionalidades visibles, hallazgos iniciales y riesgos preliminares.

Agenda

- ➊ Ficha técnica del sistema
- ➋ Mapa general de arquitectura
- ➌ Inventario funcional
- ➍ Hallazgos iniciales
- ➎ Matriz preliminar de riesgos
- ➏ Conclusiones

¿Qué es EIEInfo?

EIEInfo es un sistema institucional de la Escuela de Ingeniería Eléctrica de la UCR.

Centraliza procesos:

- académicos,
- administrativos,
- de profesores,
- de estudiantes,
- de egresados.

Lectura general

Es un sistema grande, activo y funcional, con señales visibles de crecimiento orgánico.

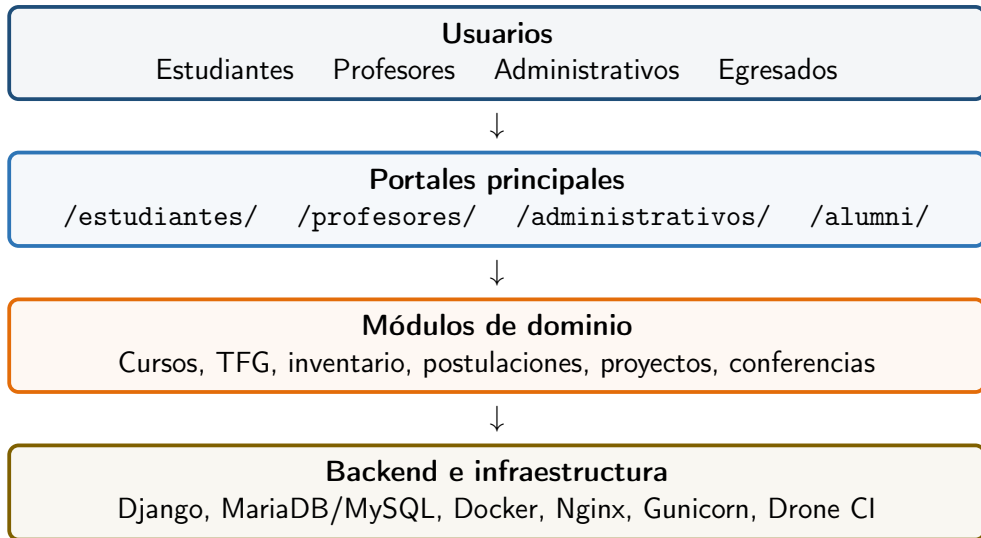
Ficha técnica del sistema

Elemento	Descripción
Framework / Lenguaje	Django 4.1.3 / Python
Base de datos	MariaDB / MySQL
Servidor web	Nginx + Gunicorn
Despliegue	Docker Compose + Drone CI
Arquitectura	Monolito modular Django, patrón MVT
Portales principales	Estudiantes, Profesores, Administrativos, Alumni
Integraciones externas	Google API, Facebook SDK, Instagram, reCAPTCHA, SMTP, Imgur

Análisis del levantamiento

Ecosistema tecnológico moderno con múltiples integraciones, pero su evolución orgánica priorizó la acumulación de funciones sobre la cohesión del diseño y la seguridad.

Mapa general del sistema



Puntos de entrada importantes

Ruta	Componente	Función principal
/estudiantes/	Portal estudiantes	Procesos estudiantiles
filapar /profesores/	Portal profesores	Gestión docente y académica
/administrativos/	Portal administrativo	Gestión operativa
filapar /alumni/	Portal egresados	Bolsa de empleo y alumni
/firma_digital/	Firma digital	Documentos formales
filapar /wiki/	Wiki institucional	Información interna
/	Página pública	Sitio visible al público

Interpretación

Las rutas muestran que el sistema se organiza por tipo de usuario y por funcionalidades institucionales específicas.

Organización por capas

Capa de portales

estudiantes, profesores, administrativos, alumni

Capa de dominio

cursos, trabajos_finales, trabajo_final_de_graduacion, postulaciones, inventario, proyectos, conferencias

Capa de soporte

firma_digital, anuncios, eventos, atributos, webpage, wiki

Núcleo técnico

settings.py, urls.py, configuración, dependencias, credenciales e integraciones externas.

Lectura arquitectónica

Sistema centralizado en infraestructura, pero fragmentado funcionalmente por portales y módulos.

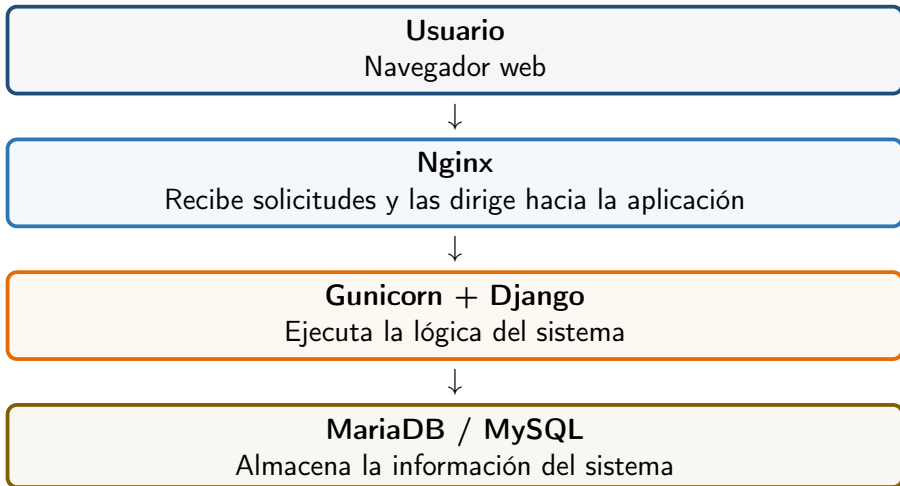
Módulos principales del sistema

Módulo	Tipo	Responsabilidad visible
profesores	Portal	Cursos, asistencias, publicaciones, comisiones
filapar administrativos	Portal	Noticias, inventario, reservaciones, estudiantes
estudiantes	Portal	Cursos, trámites, TFG, práctica profesional
filapar alumni	Portal	Egresados y bolsa de empleo
cursos	Dominio	Cursos, grupos y plan de estudios
filapar inventario	Dominio	Bienes, activos y préstamos
firma_digital	Soporte	Firma y notificación de documentos
filapar	Dominio	Flujo nuevo de TFG
trabajo_final_de_graduacion		

Observación

Profesores y administrativos concentran varias responsabilidades: candidatos prioritarios para análisis posterior.

Visión general de despliegue



Docker Compose levanta estos componentes como servicios; Drone CI automatiza el despliegue.

Actores identificados

- Estudiantes
- Profesores
- Administrativos
- Alumni
- Visitantes públicos
- Sistema / cron jobs

Flujos principales

- Gestión de cursos
- Asistencias
- Trabajo Final de Graduación
- Práctica profesional
- Firma digital
- Inventario
- Bolsa de empleo

Módulos críticos

Profesores, administrativos, cursos, TFG, firma digital e inventario.

Los hallazgos se agrupan en tres áreas:

- ❶ Seguridad y configuración
- ❷ Diseño y arquitectura
- ❸ Mantenibilidad y deuda técnica

Hallazgos de seguridad y configuración

ID	Título	Descripción / evidencia	Impacto	Crit.
H-01	DEBUG=True por defecto	Configuración inicial activa modo debug y depende del hostname para desactivarse.	Exposición de errores, rutas internas y datos sensibles.	Crítica
filapar H-02	Credenciales versionadas	Archivo de credenciales dentro del repositorio.	Posible exposición de secretos y claves API.	Crítica
H-03	Contraseña SMTP en código	Contraseña de correo definida en configuración.	Riesgo de uso indebido del correo institucional.	Crítica
filapar H-05	ALLOWED_HOSTS permisivo	Uso de ['*'] fuera del ambiente esperado.	Configuración insegura en despliegues.	Alta

Cómo explicarlo

Los riesgos más urgentes no están en la lógica funcional, sino en la configuración y manejo de secretos.

Hallazgos de diseño arquitectónico

ID	Título	Descripción / evidencia	Impacto	Crit.
H-04	Autenticación híbrida	Cada portal maneja claves de sesión propias.	Permisos difíciles de auditar y revocar.	Alta
filapar H-06	God Object	<code>consejo_asesor.py</code> concentra demasiada lógica.	Cambios más riesgosos y difíciles de probar.	Alta
H-07	Duplicación del dominio TFG	Coexisten dos módulos de trabajos finales.	Ambigüedad e inconsistencia funcional.	Alta
filapar H-08	Administrativos sobrecargado	El módulo agrupa áreas funcionales distintas.	Baja cohesión y mayor costo de mantenimiento.	Alta
H-09	Fork sin versión fija	Dependencia directa a <code>main</code> de <code>django-wiki</code> .	Instalaciones futuras pueden romperse.	Alta

Cómo explicarlo

Estas partes del sistema acumularon responsabilidades con el tiempo, sin una refactorización planificada.

Hallazgos de mantenibilidad

ID	Título	Descripción / evidencia	Impacto	Crit.
H-10	Sin pruebas activas	trabajos_finales/tests.py vacío o comentado.	Cambios sin red de seguridad.	Media
filapar H-11	Módulos zombi	Directorios con poca o nula implementación activa.	Confusión sobre qué está vigente.	Media
H-12	Zona horaria incorrecta	Uso de America/Guatemala.	Ambigüedad en sistema costarricense.	Baja
filapar H-13	Templates duplicados	Plantillas similares en varias ubicaciones.	Correcciones deben repetirse.	Media
H-14	Dependencias obsoletas	Librerías antiguas o deprecadas.	Vulnerabilidades e incompatibilidad.	Media
filapar H-15	Encoding corrupto	Caracteres dañados en configuración.	Baja legibilidad del código.	Baja

Cómo explicarlo

No detienen el sistema hoy, pero aumentan el costo de mantenerlo y evolucionarlo.

Resumen de los 15 hallazgos

Críticos

- H-01 DEBUG=True
- H-02 Credenciales
- H-03 SMTP en código

Altos

- H-04 Autenticación
- H-05 Hosts permisivos
- H-06 God Object
- H-07 TFG duplicado
- H-08 Administrativos
- H-09 Wiki sin fijar

Medios / bajos

- H-10 Sin pruebas
- H-11 Módulos zombi
- H-12 Zona horaria
- H-13 Templates
- H-14 Dependencias
- H-15 Encoding

Prioridad

Primero seguridad y configuración; después autenticación; finalmente refactorización.

Matriz preliminar de riesgos

Riesgo	Causa probable	Impacto	Prob.	Prior.
Exposición de credenciales	Secretos versionados	Crítico	Alta	P1
filapar Información sensible por debug	DEBUG=True por defecto	Crítico	Media	P1
Autorización inconsistente	Sesiones paralelas por portal	Alto	Media	P2
filapar Ruptura por dependencia no fijada	Fork en rama main	Alto	Alta	P2
Inconsistencia entre módulos TFG	Duplicación de dominio	Alto	Media	P2
filapar Cambios riesgosos en módulos grandes	Alto acoplamiento	Alto	Alta	P2
Cambios sin cobertura	Pruebas incompletas	Medio	Alta	P3
filapar Dependencias obsoletas	Librerías antiguas	Medio	Alta	P3

Lectura de la matriz

Los riesgos P1 deben atenderse antes de cualquier refactorización profunda.

¿Centralizado o fragmentado?

Centralizado en infraestructura y configuración, pero fragmentado funcionalmente por portales.

¿Qué módulos concentran más responsabilidad?

Profesores y administrativos; especialmente las vistas relacionadas con consejo asesor.

¿Qué partes parecen activas y cuáles legadas?

Activas: firma digital, TFG nuevo, postulaciones. Potencialmente legadas: trabajos finales antiguo y módulos incompletos.

¿Dónde hay señales de deuda técnica?

En módulos grandes, dependencias antiguas, templates duplicados, configuraciones sensibles y coexistencia de componentes viejos y nuevos.

- EIEInfo es un sistema grande, funcional y relevante para la operación de la Escuela.
- Su arquitectura predominante es un monolito modular Django.
- Los principales riesgos iniciales están en seguridad, configuración y mantenibilidad.
- Hay señales claras de crecimiento orgánico y deuda técnica acumulada.
- La recomendación inicial es contener primero los riesgos críticos y luego planificar refactorizaciones controladas.

Cierre

El sistema tiene una base técnica rescatable, pero requiere estabilización antes de evolucionar.

Gracias

Auditoría de Diseño de Software
Sistema EIEInfo

Mariana Bernal Hernández — C31121